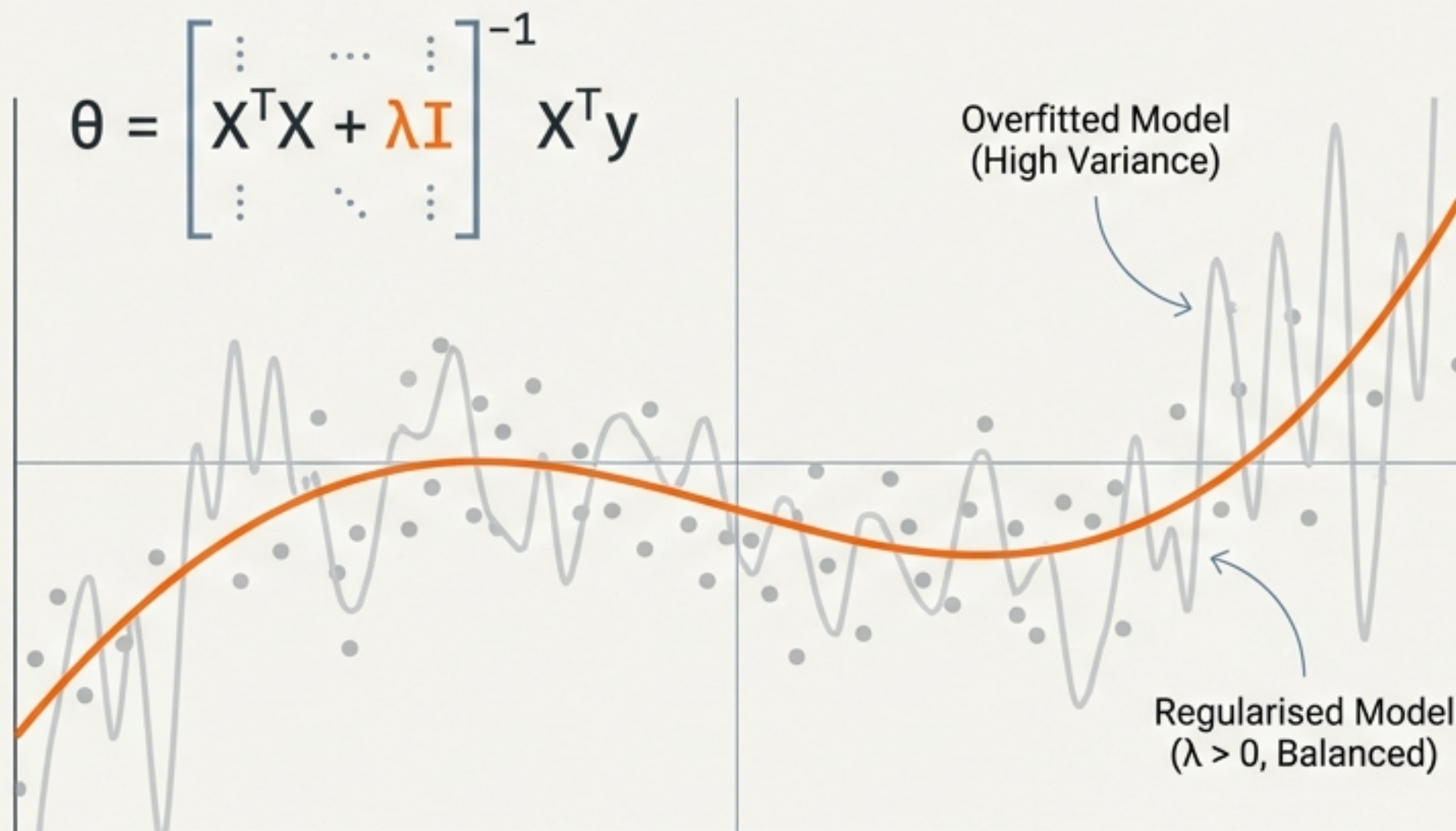


Ridge Regression From Scratch

Mathematical Formulation
& Python Implementation

$$J(\theta) = \text{MSE}(\theta) + \left[\lambda \sum \theta_i^2 \right]$$

Regularisation adds a penalty term to the loss function to prevent overfitting, controlled by the hyperparameter λ .



From 1D Intuition to N-Dimensional Matrix Calculus

```
import numpy as np

def ridge_regression(X, y, lambda_val):
    # Add bias term
    X_b = np.c_[np.ones((len(X), 1)), X]
    n_features = X_b.shape[1]
    # Regularisation matrix
    reg_matrix = lambda_val * np.eye(n_features)
    reg_matrix[0, 0] = 0 # Do not regularise bias

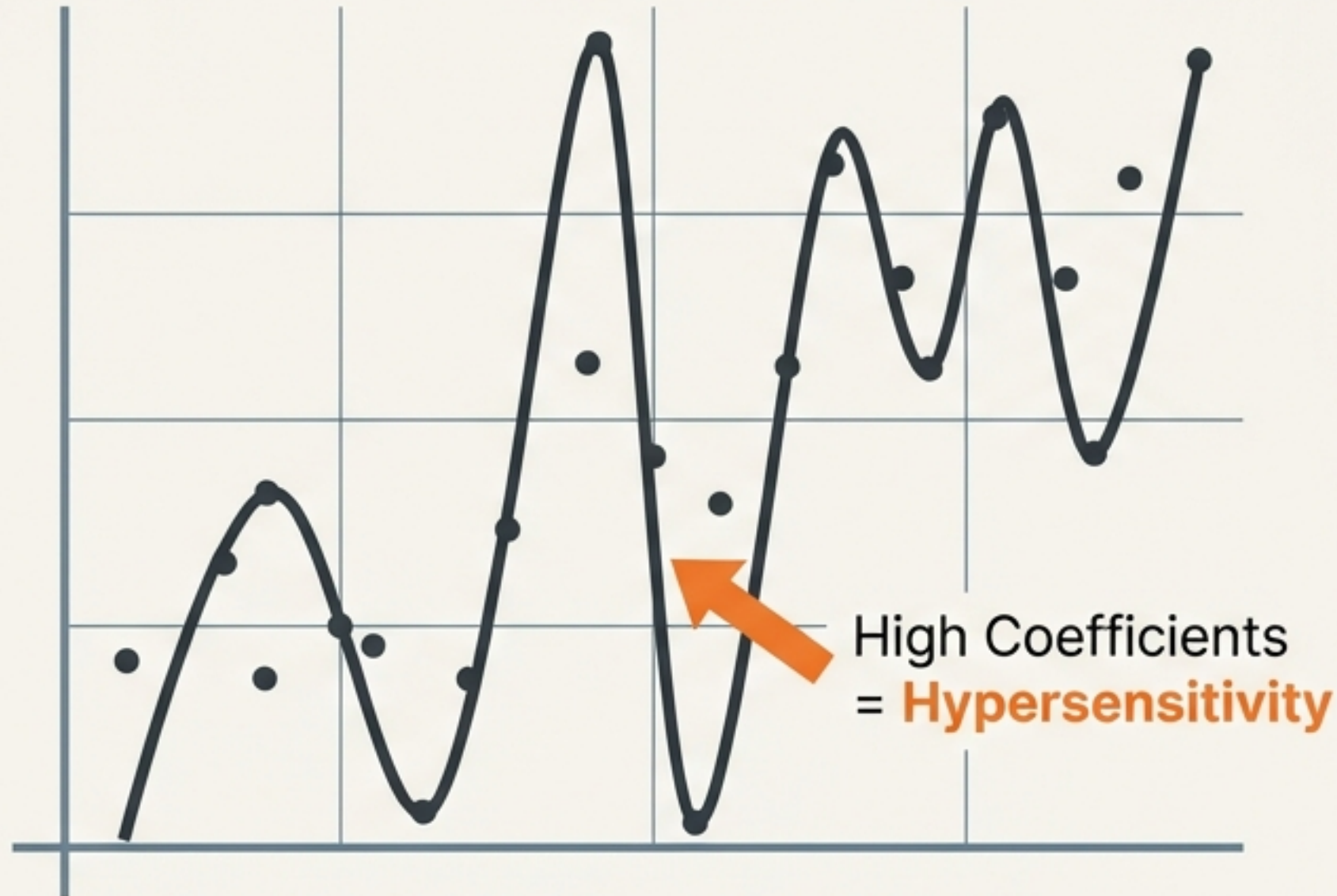
    # Closed-form solution
    theta = np.linalg.inv(X_b.T @ X_b + reg_matrix) @ X_b.T @ y
    return theta
```

Python code for closed-form solution, incorporating the Tangerine penalty λ .

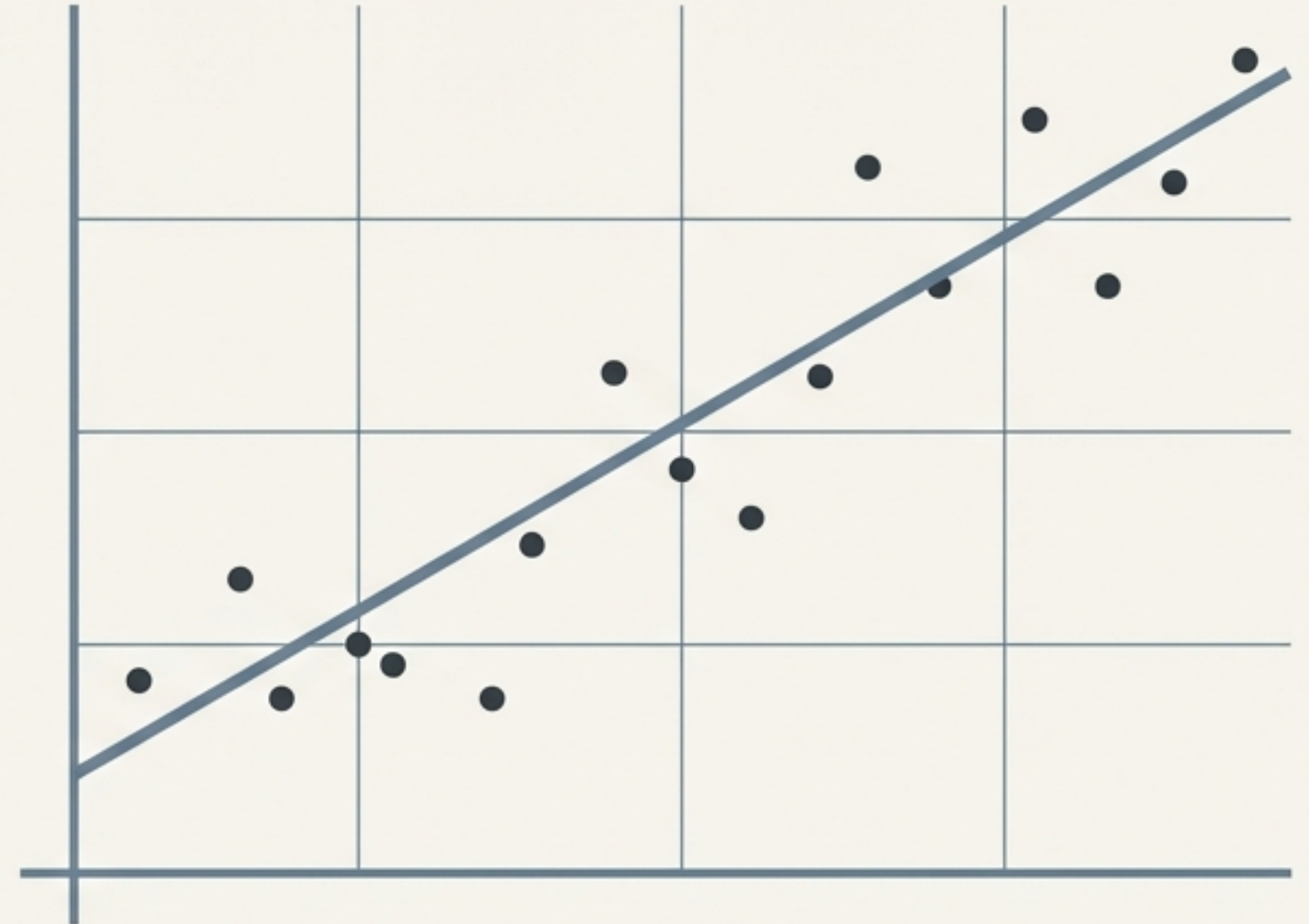
```
# Example Usage
# theta_ridge = ridge_regression(X, y, lambda_val=1.0)
```


The Core Problem: Overfitting in Linear Regression

Overfitting: High Variance

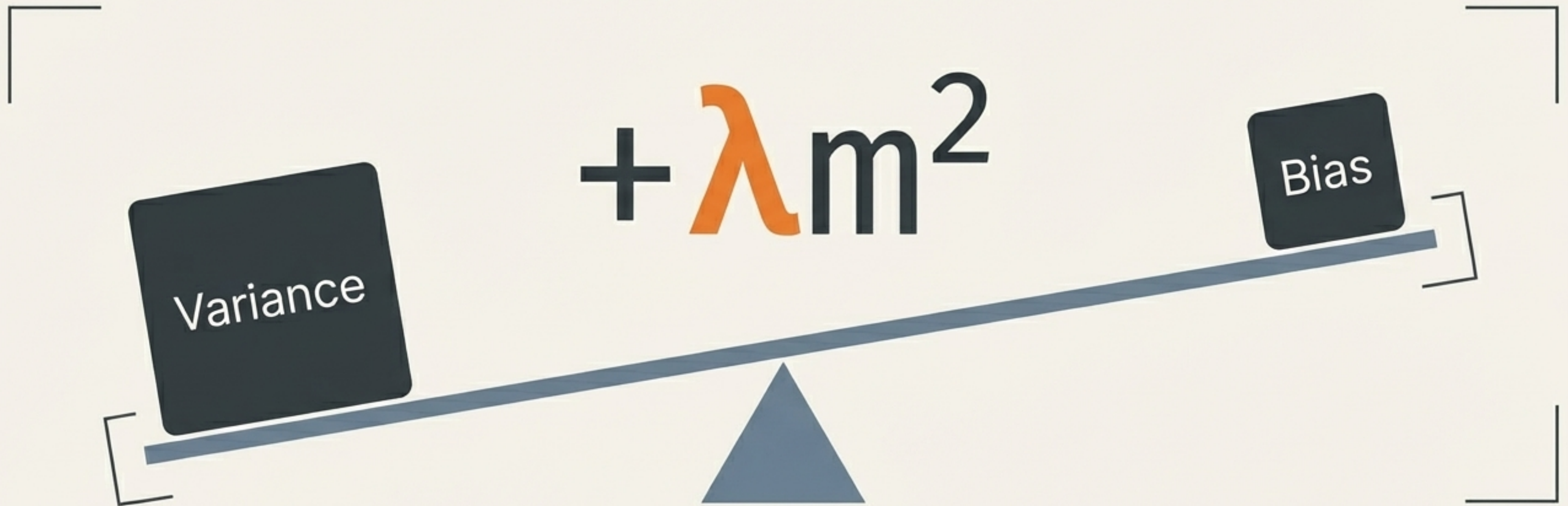


When the slope (m) is excessively high, Y becomes overly dependent on X .



This leads to models that memorise training data but fail on unseen data (High Variance).

The Solution: Regularisation & The Penalty Term



- By artificially adding a penalty term to our cost function, we restrict the slope's growth.
- We accept a slight increase in training error (bias) in exchange for a massive reduction in variance.

Formulating the 1D Mathematics

$$L = \underbrace{\sum (y_i - (mx_i + b))^2}_{\text{Standard OLS Cost Function}} + \underbrace{\lambda m^2}_{\text{L2 Regularisation Penalty}}$$

Instead of just minimising the error, the algorithm must now minimise the error
AND the magnitude of the slope (m).

λ acts as the hyperparameter controlling the penalty strength.

Deriving the Shrinkage Effect

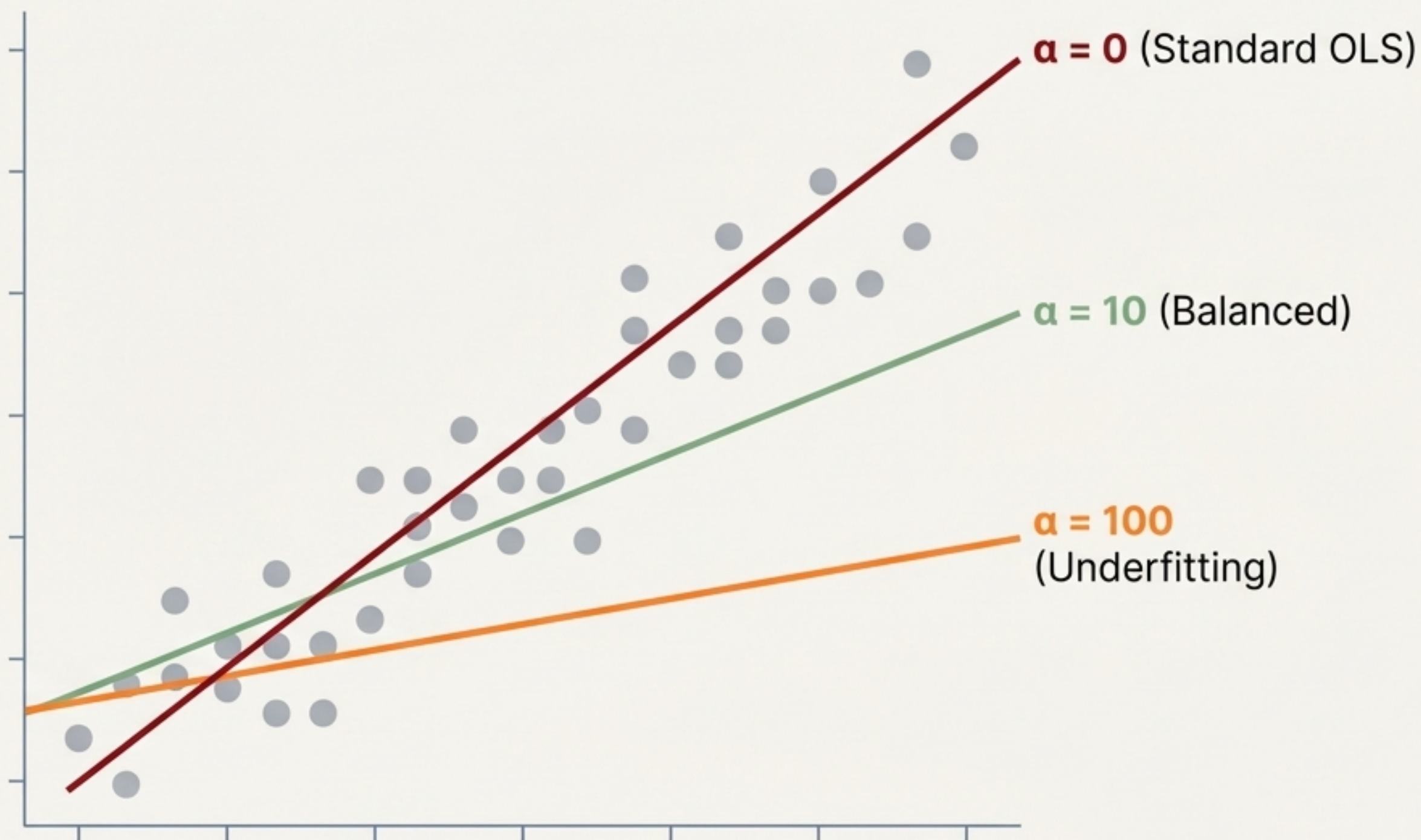
$$\frac{\partial L}{\partial m} = 0$$

$$-2 \sum x_i (y_i - (mx_i + b)) + 2\lambda m = 0$$

$$m = \frac{\sum (y_i - \bar{y})(x_i - \bar{x})}{\sum (x_i - \bar{x})^2 + \lambda}$$

Because λ is directly added to the denominator, any increase in λ mathematically forces the slope (m) to shrink.

Visualising the Impact of the Hyperparameter



- As α scales from 0 to 100, the regression line progressively flattens.
- Finding the optimal α is the key to perfectly balancing the model.

(Note: α is Scikit-Learn's programmatic equivalent for the mathematical term λ)

Scaling to N-Dimensions: Matrix Representation

$$\begin{array}{c} \text{Prepended} \\ \text{column of 1s} \end{array} \rightarrow \begin{array}{c} \mathbf{X} \\ \left[\begin{array}{cccccc} 1 & x_{11} & x_{12} & x_{13} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & x_{23} & \dots & x_{2n} \\ 1 & x_{31} & x_{32} & x_{33} & \dots & x_{3n} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & x_{m3} & \dots & x_{mn} \end{array} \right] \end{array} \begin{array}{c} \mathbf{W} \\ \left[\begin{array}{c} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{array} \right] \end{array} \leftarrow \begin{array}{l} \text{Intercept term} \\ \text{absorbed into} \\ \text{weight vector} \end{array}$$

To handle multiple features, we transition to matrices. Prepending a column of 1s to our input matrix X allows us to absorb the intercept term (w_0) directly into our weight vector W .

The N-Dimensional Cost Function

Matrix equivalent
of squared errors



$$L = (XW - Y)^T(XW - Y) + \lambda W^T W$$

Matrix equivalent
of $\sum \lambda m^2$



The core logic remains identical to 1D, completely rewritten using linear algebra to calculate all weights simultaneously.

Matrix Differentiation

Expand:

$$L = W^T X^T X W - 2W^T X^T Y + Y^T Y + \lambda W^T W$$

Differentiate w.r.t **W** & Set to 0:

$$2X^T X W - 2X^T Y + 0 + 2\lambda W = 0$$

Group Terms:

$$X^T X W + \lambda W = X^T Y$$

Applying standard matrix calculus rules to find the gradient. Setting the derivative to zero allows us to isolate the weight matrix **W**.

The Closed-Form Solution

$$W = (X^T X + \lambda I)^{-1} X^T Y$$


Identity Matrix
($n+1 \times n+1$)

This single equation calculates the optimal weights and intercept in one calculation. The addition of the Identity Matrix (I) ensures the matrix is always invertible.

From Math to Python: Building the Class

```
class RidgeRegression:
    def __init__(self, alpha=0.1):
        self.alpha = alpha

    def fit(self, X_train, Y_train):
        X_train = np.insert(X_train, 0, 1, axis=1)
        # linear algebra implementation follows...
```



Prepending the column of 1s to absorb the intercept, exactly as mapped in our math.

Translating the Equation to NumPy

$$W = (X^T X + \lambda I)^{-1} X^T Y$$

```
W = np.linalg.inv(np.dot(X.T, X) + alpha * I).dot(X.T).dot(Y)
```

Direct translation requires zero loops, leveraging highly optimised linear algebra libraries.

The Validation Discrepancy

Scikit-Learn Library

Coefficient: 44.02

Intercept: 185.90



Our Custom Class

Coefficient: 44.00

Intercept: 184.80

When validated against Scikit-Learn's official Ridge class, the numbers are extremely close, but not identical.

Why is our mathematically sound derivation producing different results?

The Scikit-Learn Gotcha (The Intercept Fix)

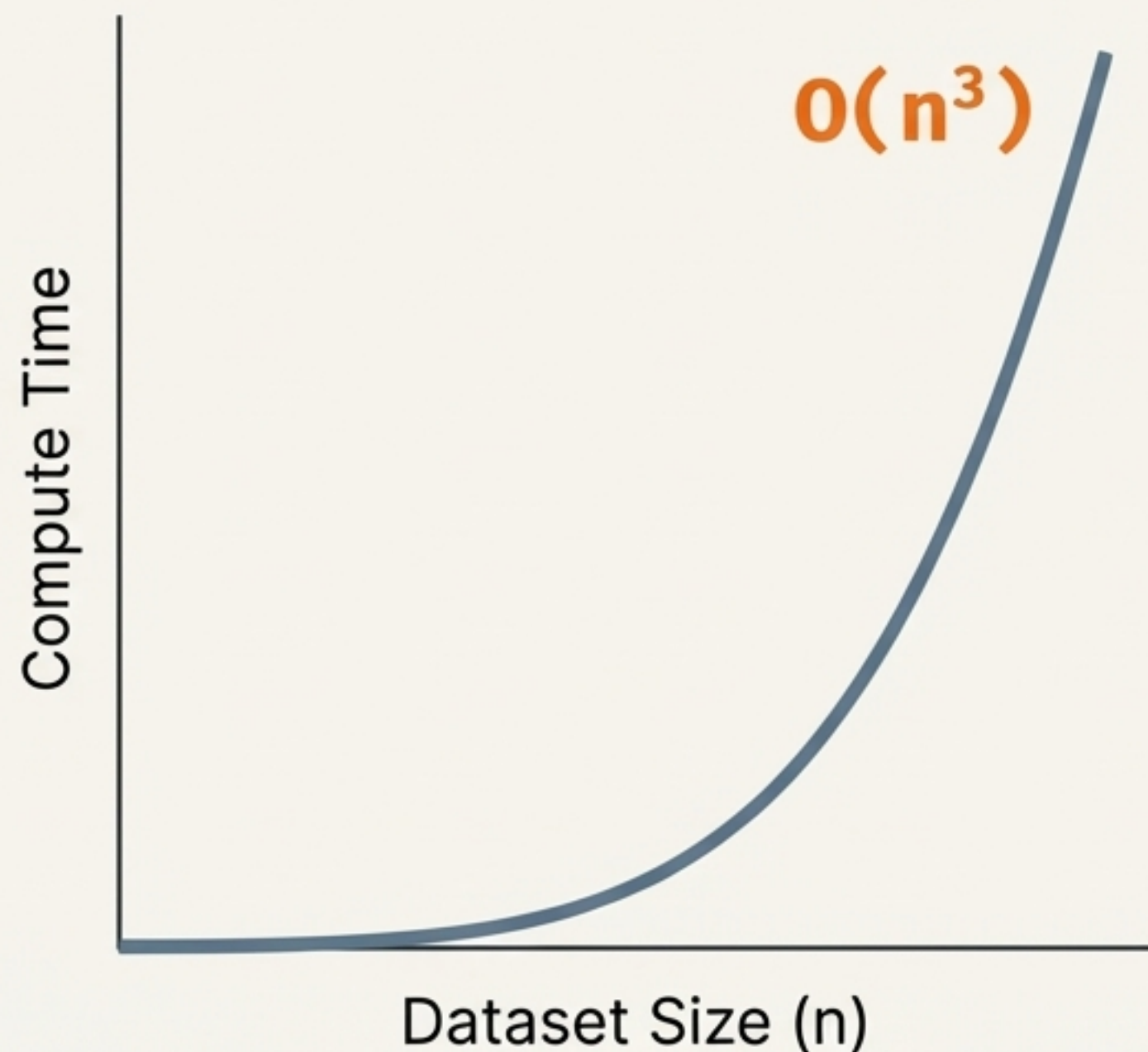
```
1 def __in__name(as_train):  
2     """expect.target[1],  
3     """  
4  
5     I = np.identity(X_train.shape[1])  
6     I[0][0] = 0  
7  
8     Wx_trainh = np.amsetup(X_mains(1)(X + * I)(Y)  
9     W.I = np.identity(X_train.shape[1], λI * I)  
10    return W_iow.I  
11
```

**We do not penalise
the intercept!**



- Scikit-Learn intentionally avoids applying the L2 penalty to the intercept term (W_0).
- By setting the first element of our Identity Matrix to zero, we remove the penalty for W_0 .
- The output now flawlessly matches Scikit-Learn down to the final decimal.

Limitations & The Road Ahead



Direct matrix inversion works flawlessly for small to medium datasets. However, the $O(n^3)$ time complexity makes it computationally crippling for massive datasets.

The Next Step: To handle big data, we must abandon the closed-form solution and embrace Gradient Descent.